Java OCA – Chapter 6 - Übungen zu Statischen Methoden

1 Binärzahl oder nicht?

Schreiben Sie eine statische Methode `boolean istBinaerZahl(String)`, die prüft, ob ein gegebener String nur aus den Zeichen 0 und 1 besteht, also eine Binärzahl repräsentiert.

2 Körperoberfläche

Die Körperoberfläche, kurz KOF, ist definiert als die äußere Oberfläche des Körpers, die von Haut bedeckt ist. Ihre Größe wird als Flächenmaß im m^2 angegeben und dient der Berechnung von Infusions- oder Chemotherapiedosen sowie der Einschätzung von Verbrennungen.

Die Körperoberfläche eines Menschen wird nach der Formel von Mosteller wie folgt errechnet:

$$KOF = \sqrt{\frac{h \cdot w}{3600}}$$

In der angegebenen Formel bezeichnet *h* die Körpergröße in cm und *w* das Körpergewicht in kg. Das Ergebnis ist die Köperoberfläche in m^2.

Aufgabenstellung

Schreiben Sie eine Funktion mit Namen `berechneKof`, die die Körperoberfläche in m^2 berechnet.

Testfall

180 cm und 58 kg → 1.7029387 m^2.

Tipps

- Beim Wurzelziehen können Sie auf `Math.sqrt( )` zurückgreifen.
- Achten Sie bei Divisionen immer darauf, dass die Nenner aus Kommazahlen bestehen.
- Bei Divisionen von Integer-Werten folgt ein Integer-Ergebnis.

Java OCA – Chapter 6 - Übungen zu Statischen Methoden

3 Die indigene Bevölkerung Nordamerikas

Die Insel Manhatten gehörte einst zum Siedlungsgebiet der Delaware-Indianer. Bis heute erinnert ein Stein daran, wie die Delaware diese Insel an den niederländischen Kaufmann Peter Minuit verkauften.
Sie verkauften die Insel im Jahr 1627 für 24 Dollar.

Auf welchen Betrag wären diese 24 Dollar in 10, 20, 50 oder 100 Jahren angewachsen, wenn sie jährlich mit 3 % verzinst worden wären und nie etwas abgehoben wurde?

Die benötigte Formel lautet

$$Endkapital = Verkaufspreis \cdot \left(1 + \frac{Zinssatz}{100}\right)^n$$

wobei der Exponent n die Laufzeit in Jahren ist.

Wie programmiert man das, wenn man keine Ahnung von statischen Methoden hat?

(siehe das Listing auf der nächsten Seite)

```
public class Zinseszins {

     public static void main(String[] args) {
          double verkaufspreis = 24;
          double zinssatz = 3;
          double laufzeit;
          double endkapital;
     //Berechnung des Kapitals nach n Jahren
     System.out.println();
     endkapital = verkaufspreis * Math.pow((1 +
zinssatz/100),10);
     System.out.println(" Nach 10 Jahren: " +(int)endkapital +
" Dollar");

     endkapital = verkaufspreis * Math.pow((1 +
zinssatz/100),20);
     System.out.println(" Nach 20 Jahren: " +(int)endkapital +
" Dollar");


     endkapital = verkaufspreis * Math.pow((1 +
zinssatz/100),50);
     System.out.println(" Nach 50 Jahren: " +(int)endkapital +
" Dollar");


     endkapital = verkaufspreis * Math.pow((1 +
zinssatz/100),100);
     System.out.println(" Nach 100 Jahren: " +(int)endkapital
+ " Dollar");


     }

}
```

Das Programm läuft. Hier gibt es kein Problem. Aber ein anderes Problem gibt es trotzdem: nämlich den Quelltext. Jeder Programmierer der über fortgeschrittene Programmierkenntnisse verfügt, wird beim Anblick dieses Programms den Kopf schütteln.

Hier wird eine wiederkehrende Aufgabe (die Zinsberechnung) jedes Mal neu implementiert. Das macht das Programm äußerst unübersichtlich und erschwert das Debuggen, spricht das Ausmerzen von Fehlern. Wenn sich jetzt ein Fehler in der Formel eingeschlichen hätte, müssen Sie diesen fünf Mal korrigieren. In größeren Programmen hätten Sie vielleicht sogar noch das Problem, dass Sie gar nicht mehr wissen, wo überall in Ihrem Quelltext diese Formel auftaucht.

Java OCA – Chapter 6 - Übungen zu Statischen Methoden

Eine Verbesserung des Quellcodes ist mithilfe von **statischen Methoden** möglich.

Aufgabe
Schreiben Sie eine **statische Methode**, die es ermöglicht das Endkapital in Abhängigkeit von der Laufzeit n, dem Verkaufspreis und dem Zinssatz „dynamisch“ zu berechnen.

Zusatzaufgabe
Im konkreten Beispiel wurden manche Variablen unnötigerweise als `double`-Werte deklariert. Welche Variablen können Sie auf den Datentypen Integer umstellen, damit das Programm nach wie vor korrekt funktioniert? Außer der Änderung des Datentyps soll nichts verändert werden.

Java OCA – Chapter 6 - Übungen zu Statischen Methoden

4 Verschlüsselung nach Vernam

Dieses Verfahren geht davon aus, dass Sender- und Empfänger das Schlüsselwort kennen und dass jedes Schlüsselwort nur einmal verwendet wird. Dann wird wie folgt verschlüsselt: Schreiben Sie unter den Klartext das Schlüsselwort. Zählen Sie dann den ASCII-Code der übereinanderstehenden Buchstaben zusammen und bilden Sie den Rest, der bei der Division durch 256 bleibt. Die entstehende Zahl ist der ASCII-Code des entsprechenden Buchstabens im Geheimtext.

Beispiel:

Klartext	Ö	l	k	ä	n	n	c	h	e	n
ASCII-Wert	214	108	107	228	110	110	99	104	101	110
Schlüsselwort	g	e	h	e	i	m	c	o	d	e
ASCII-Wert	103	101	104	101	105	109	99	111	100	101
Summe der ASCII-Werte	317	209	211	329	215	219	198	215	201	211
Rest mod 256	61	209	211	73	215	219	198	215	201	211
Geheimtext (Windows)	=	Ñ	Ó	I	×	Û	Æ	×	É	Ó

Aufgabe

Entwickeln Sie eine statische Methode namens `chiffriere`, die den Klartext und das Schlüsselwort als Parameter erhält und den Geheimtext als Funktionswert zurückgibt. Testen Sie Ihr Programm, indem Sie in der `main( )`−Methode die Methode aufrufen und den Geheimtext in der Konsole ausgeben.

Tipps

- Sie dürfen – wie im Beispiel – annehmen, dass Klartext und Schlüsselwort die gleiche Anzahl von Zeichen haben.
- Falls Sie stocken und nicht mehr weiterkommen: Versuchen Sie zunächst für ein einzelnes Zeichen zur Lösung zu kommen. Wenn Ihnen das gelungen ist, müssen Sie Ihr Programm nur noch um eine `for`- oder `while`-Schleife ergänzen.
- Die Java String Klasse `charAt( )` gibt einen `char` Wert an der gegebenen Indexnummer zurück.

Java OCA – Chapter 6 - Übungen zu Statischen Methoden

5 Schaltjahr Prüfung

Julius Caesar hatte einen Kalender eingeführt, der 365 Tage hat. Solange braucht nämlich die Erde um die Sonne 1-mal zum umlaufen. Dummerweise sind es nicht genau 365 Tage, sondern etwa 6 Stunden mehr. Pro Jahr. Das bedeutet, dass man irgendwann Ostern im Sommer oder Herbst feiern würde, berücksichtigte man nicht diese 5 Stunden Zeitdifferenz.

Überschlagen wir mal kurz: Wenn pro Jahr ein Fehler von etwa 6 Stunden gemacht wird, wann haben wird dann einen Fehler von 24 Stunden, also von 1 Tag?

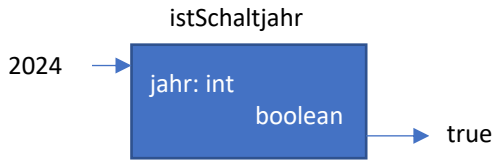
$$x * 6\frac{h}{Jahr} = 24\,h \implies 4\ Jahre$$

Deswegen haben wir alle 4 Jahre ein sogenanntes Schaltjahr. Bei Schaltjahren hat der Februar 29 Tage anstatt der üblichen 28 Tage. Es gibt aber nicht nur diese Regel. Weniger bekannt sind die weiteren Regeln, die bestimmen, wann es einen Schalttag geben soll oder eben nicht.

- alle 100 Jahre soll es keinen Schalttag geben.
- alle 400 Jahre hingegen 1 Schalttag

Zusammenfassung:
Schaltjahre können Sie dadurch bestimmen, wenn die **Jahreszahl durch 4** teilbar ist. Allerdings darf die Jahreszahl **zusätzlich nicht durch 100 teilbar** sein. Wenn die Jahreszahl wiederum **durch 400 teilbar** ist, ist es doch ein Schaltjahr.

Aufgabe
Schreiben Sie eine Funktion, die überprüft, ob es sich bei einer übergebenen Jahreszahl um ein Schaltjahr handelt. Sollte das Jahr ein Schaltjahr sein, wird `true` zurückgegeben. Sollte dies nicht der Fall sein, wird `false` zurückgegeben.



Java OCA – Chapter 6 - Übungen zu Statischen Methoden

Testfälle

- Schaltjahre
  - 2016
  - 2024
  - 1916
- Keine Schaltjahre
  - 2013
  - 2003
  - 1900

Algorithmische Tipps

Wenn Sie stocken und nicht mehr weiterwissen, dann versuchen Sie mal Folgendes:

- Probieren Sie, die Abläufe in der Funktion testweise auszugeben. Wird zum Beispiel bei einer entsprechenden Jahreszahl die erwartete Stelle im Quellcode ausgeführt? Sollte dies nicht der Fall sein, scheint es einen (Denk-)-Fehler im Quellcode zu geben.
- Teilbarkeit in Zahlen lässt sich mit dem Modulo-Operator (%) prüfen, welcher den Rest einer Division zurückgibt.

6 Überladen von Methoden

Können die beiden Funktionen `quadrat()` in derselben Klasse stehen? Falls ja, warum?

```
public static int quadrat (int zahl)
{
     return zahl * zahl;
}
public static double quadrat(double zahl)
{
     return zahl * zahl;
}
```

Java OCA – Chapter 6 - Übungen zu Statischen Methoden

7 Body-Mass-Index

Der Body-Mass-Index (BMI) ist eine Zahl, die die Relation von Körpergewicht zu Körpergröße angibt. Berechnet wird dieser nach der Formel:

$$BMI = \frac{m}{l^2}$$

m: Körpergewicht in Kilogramm
l: Körpergröße in Metern

Anhand des BMI-Wertes kann bestimmt werden, in welche Gewichtskategorie eine Person fällt. Hierfür werden in dieser Aufgabe folgende Abstufungen verwendet:

- < 18.5: untergewichtig
- 18.5 – 25: normalgewichtig
- 25-30: übergewichtig
- > 30: fettleibig

Aufgabenstellung

Schreiben Sie eine statische Methode in der Klasse `Health`, die den BMI für ein gegebenes Körpergewicht und die Körpergröße berechnet.

Fügen Sie der Klasse `Health` eine zweite statische Methode hinzu, die für einen gegeben BMI zurückgibt, in welche Gewichtskategorie dieser fällt.

Schreiben Sie in einer separaten Klasse ein Programm, dass das Körpergewicht und die Körpergröße einliest und mit der vorher verfassten Methode `Health.computeBMI( )` den BMI berechnet. Bestimmen Sie anschließend mit dem BMI-Wert die Gewichtskategorie und konstruieren Sie folgende Konsolenausgabe:

```
Mit einem BMI von <BMI> sind Sie <Kategorie>.
```

Die spitzen Klammern sollen mit den entsprechenden Werten ersetzt werden.

Testfälle

- Gewicht: 57 kg, Größe: 1.80 m
  - `Mit einem BMI von 17.6 sind Sie untergewichtig.`
- Gewicht: 81 kg, Größe: 1.80 m
  - `Mit einem BMI von 25.0 sind Sie normalgewichtig`
  - oder `Mit einem BMI von 25.0 sind Sie übergewichtig.`
- Gewicht: 120 kg, Größe: 2.00 m
  - `Mit einem BMI von 30.0 sind Sie übergewichtig.`

Java OCA – Chapter 6 - Übungen zu Statischen Methoden

8 GPS-Luftlinie

Wie weit liegen zwei Orte voneinander entfernt? Für solche Fragen können wir das Global Positioning System (GPS) zu Hilfe nehmen. Mit den Daten des Systems wollen wir die Distanz zwischen zwei GPS-Koordinaten berechnen.

Zunächst aber müssen wir Zahlen vom Gradmaß (engl. degree) ins Bogenmaß (engl. radian) umrechnen.

Das Bogenmaß berechnet sich wie folgt aus dem Gradmaß:

$$radian = \frac{degree}{180} \cdot \pi$$

Die Kreiszahl π steht als globale `double`-Variable unter dem Namen `PI` bereit.
Eine GPS-Koordinate wird in Breitengrad (engl. latitude) und Längengrad (engl. longitude) angegeben, wobei beide Gradzahlen vom Typ `float` sind.

Die Entfernung d berechnet sich nach der folgenden Formel:

$$c = \mathrm{acos}(\sin(lat1) \cdot \sin(lat2) + \cos(lat1) \cdot \cos(lat2) \cdot \cos(lon2 - lon1))$$

$$d = c \cdot 6378{,}137$$

Die Variablen `lat1` und `lon1` bezeichnen den Breiten- und Längengrad der ersten GPS-Koordinate. Entsprechendes gilt für die zweite GPS-Koordinate und Variablen `lat2` und `lon2`. Diese Zahlen sind im Gradmaß angegeben. Die in der Berechnung verwendeten trigonometrischen Funktionen *sin( )*, *cos( )* und *acos( )* benötigen im Argument Angaben im Bogenmaß.

Aufgabenstellung

Schreiben Sie eine statische Methode mit dem Namen `toRadians`, die eine Zahl vom Gradmaß ins Bogenmaß umrechnet.

Geben Sie dann eine zweite statische Methode mit dem Namen `gpsDistanz` an, die die Entfernung d zwischen zwei GPS-Koordinaten berechnet. Verwenden Sie zur Umrechnung zwischen Gradmaß und Bogenmaß die vorher implementierte `toRadians`-Methode.

Java OCA – Chapter 6 - Übungen zu Statischen Methoden

Testfälle

- Die Entfernung zwischen dem Kölner Dom (Breitengrad: 50.94157, Längengrad: 6.95821) um dem Düsseldorfer Fernsehturm (Breitengrad: 51.21795, Längengrad: 6.76165) beträgt 33,687 km Luftlinie.
- Zwischen der Hamburger Elbphilharmonie (Breitengrad: 53.54125, Längengrad: 9.9841) und der Münchner Frauenkirche (Breitengrad:48.13663 Längengrad: 11.57715) sind es 611,928 km Luftlinie.

Algorithmische Tipps

Wenn Sie stocken und nicht mehr weiterwissen, dann versuchen Sie mal Folgendes:

- Für die trigonometrischen Funktionen *acos( ), sin( )* und *cos( )* stehen uns gleichnamige Funktionen bereit, die auf dem Bogenmaß definiert sind.
- Bei Divisionen immer darauf achten, welche Variablentypen miteinander dividiert werden.

Java OCA – Chapter 6 - Übungen zu Statischen Methoden

9 Partnervermittlung

Viele Partnervermittlungsdienste gleichen zum Partnermatching Interessenprofile ab. Dazu werden im Vorfeld die Interessen über einen Fragebogen abgefragt. Ein Interessenprofil wird im Kontext dieser Aufgabe als `boolean`-Array repräsentiert. Die Elemente des Arrays stellen die nachgefragten Interessen dar. Die Wahrheitswerte im Array besagen, ob ein Interesse besteht (`true`) oder nicht (`false`).

Aufgabenstellung

Schreiben Sie eine statische Methode mit dem Namen `interestMatch`, die zwei Interessenprofile abgleicht und die prozentuale Übereinstimmung als Ganzzahl zurückliefert. Die statische Methode soll prüfen, ob beide Interessenprofile gleich groß sind. Wenn dem nicht so ist, soll als prozentuale Übereinstimmung 0 zurückgegeben werden.

Testfälle

- Partner A: true, true, false, false, false, true
- Partner B: true, true, false, false, false, false
- Partner C: false, false, true, true, true, false
- Partner D: true, true, false, false, false

Ergebnisse

- interestMatch(personA, personA) → 100 %
- interestMatch(personA, personB) → 83 %
- interestMatch(personA, personC) →0%
- interstMatch(personA, personD) → 0%

Algorithmische Tipps

Wenn Sie stocken und nicht mehr weiterwissen, dann versuchen Sie doch Folgendes:

- Prüfen Sie die Größen der Arrays.
- Gehen Sie alle Fragen nacheinander durch und erhöhen Sie den Zähler, wenn beide Partner die gleiche Antwort angekreuzt haben.
- Berechnen sie die Prozentzahl: $\frac{Übereinstimmungen}{Gesamtzahl} * 100.$

10 Gerade vor ungeraden Zahlen

Schreiben Sie eine statische Methode `void geradeVorUngerade( int [ ] )`. Diese soll ein gegebenes Array von `int`-Werten so umordnen, dass die geraden Zahlen zuerst auftreten und dahinter dann die ungeraden Zahlen. Die Reihenfolge innerhalb der geraden und ungeraden Zahlen ist nicht von Relevanz.

Beispiel

Eingabe	Ausgabe
[3, 5, 6, 4, 9, 5, 2]	[6, 4, 2, 5, 9, 5, 3]